Deterministic Word-Addressed CPU Interpreter with Fault-Safe Execution and Observational Cycle Accounting

A clean-room implementation of a 16-bit CPU interpreter featuring explicit architectural state management, deterministic fault handling, and precise cycle accounting.

Key Features

- **Explicit fetch–decode–execute model** with side-effect-free fetch phase.
- **Program Counter ownership** handled by instruction semantics, not centralised control.
- **Register-indirect addressing** enables full 64K-word memory access.
- **Deterministic fault handling** for division by zero and invalid opcodes.
- **Observational cycle model** decoupled from correctness logic.
- **Stress-tested** with backward jumps, infinite loops, and fault dominance scenarios.

Architecture Overview

CPU State (`CPUState`)

The CPU maintains the following architectural state:

Component	Type	Description
`GenPR[4]`	`uint16_t[4]`	General-purpose registers (R0–R3)
`pc`	`size_t`	Program counter (word-addressed)
`sf`	`uint8_t`	Status flag (zero flag)
`data[65536]`	`uint16_t[65536]`	128KB RAM (64K words)
`halted`	`bool`	Halt state indicator
`reason`	`HaltReason`	Reason for halt
`cycle_count`	`uint64_t`	Observational cycle counter

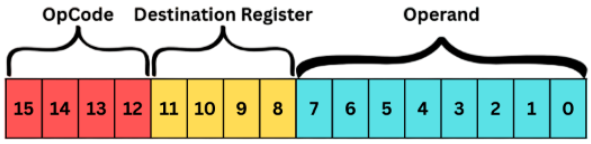
Memory Model

- **Word-addressed**: Instructions and data are 16-bit words.
- **Unified memory**: No separation between instruction and data memory.
- **Size**: 65,536 words (128KB).
- **Access**: Direct indexing via program counter and register-indirect addressing.

Instruction Set Architecture

Encoding Format

Instructions are encoded as 16-bit words:



Bits [11:8] form the destination register field. Although this field is 4 bits wide, only the lower two bits are currently decoded, allowing four general-purpose registers (R0–R3). The remaining bits are reserved for future expansion.

Supported Instructions

Opcode	Mnemonic	Format	Operation	Cycles
`0x1`	LOAD	`1dxx`	`R[d] ← imm8`	1
`0x2`	ADD	`2dss`	`R[d] ← R[d] + R[s]`	1
`0x3`	STORE	`3dxx`	`MEM[R[x]] ← R[d]`	4
`0x4`	SUB	`4dss`	`R[d] ← R[d] - R[s]`	1
`0x5`	MUL	`5dss`	`R[d] ← R[d] × R[s]`	1
`0x6`	DIV	`6dss`	`R[d] ← R[d] ÷ R[s]`	12*
`0x7`	JMP	`7xxx`	`PC ← addr12`	2
`0xF`	HALT	`Fxxx`	Halt execution	1

* Division by zero causes an immediate fault (1 cycle)

Instruction Details

LOAD (0x1)
Encoding: 0x1 d imm8
Example: 0x100A → R0 = 10

Loads an 8-bit immediate value into a register.

ADD (0x2)
Encoding: 0x2 d s
Example: 0x2001 → R0 = R0 + R1

Adds two registers and updates the status flag.

STORE (0x3)
Encoding: 0x3 d r
Example: 0x3002 → MEM[R2] = R0

Register-indirect store. Uses the value in register `r` as the memory address.

SUB (0x4)
Encoding: 0x4 d s
Example: 0x4001 → R0 = R0 - R1

Subtracts registers, updates the status flag.

MUL (0x5)
Encoding: 0x5 d s
Example: 0x5001 → R0 = R0 × R1

Multiplies registers, updates the status flag.

DIV (0x6)
Encoding: 0x6 d s
Example: 0x6001 → R0 = R0 ÷ R1

Divides registers. Triggers a fault if the divisor is zero.

JMP (0x7)
Encoding: 0x7 addr12
Example: 0x7005 → PC = 5

Unconditional jump to 12-bit absolute address.

HALT (0xF)
Encoding: 0xF000

Halts execution with a success status.

Execution Model

Fetch-Decode-Execute Cycle

1. **Fetch**: Read 16-bit instruction from `data[PC]`.
2. **Decode**: Extract opcode, destination, and operands.
3. **Execute**: Operate, update architectural state.
4. **Update PC**: Increment PC (unless instruction suppresses it).

Program Counter Management

- **Default behaviour**: PC increments by 1 after each instruction.
- **Suppression**: JMP, HALT, and fault instructions suppress the default increment.
- **Ownership**: Each instruction controls whether the PC advances.

Fault Handling

The interpreter treats faults as first-class architectural events:

Fault Type	Trigger	Behavior
`DIVISION_BY_ZERO`	Divisor is zero	Halt immediately, preserve PC
`UNKNOWN_OPCODE`	Invalid opcode	Halt immediately, preserve PC

Critical property: When a fault occurs, the architectural state reflects the moment of failure; no partial updates are committed.

Cycle Accounting

The interpreter implements observational cycle counting:

- Each instruction has a fixed cycle cost.
- A cycle counter is never used for control flow decisions.
- Provides performance metrics without affecting determinism.
- Accumulated in the `cycle_count` field.

Cycle Costs by Instruction Class

- **ALU operations** (ADD, SUB, MUL): 1 cycle.
- **Division** (successful): 12 cycles.
- **Memory operations** (STORE): 4 cycles.
- **Control flow** (JMP): 2 cycles (pipeline flush metaphor).
- **Faults/HALT**: 1 cycle.

API Reference

Core Functions

`void run(CPUState &cpu)`

Executes the program loaded in CPU memory until the halt condition.

Parameters:

- `cpu`: Reference to initialised CPU state.

Behavior:

- Prints state before each instruction.
- Executes fetch-decode-execute loop.
- Prints the final state on halt.

`void load_program(CPUState &cpu, const std::vector<uint16_t> &program)`

Loads a program into CPU memory and resets execution state.

Parameters:

- `cpu`: Target CPU state.
- `program`: Reference to a vector of 16-bit instruction words.

Side effects:

- Initialises the program to memory starting at address 0.
- Resets PC to 0.
- Clears the halt state.

`void print_cpu_state(const CPUState &cpu, bool final = false)`

Prints formatted CPU state for debugging.

Example output format:

Registers: R0:000A R1:0005 R2:0000 R3:0000
PC: 0003 | SF: 0 | Cycles: 7 | Halted: NO | Reason: NONE

Design Principles

1. Explicit State Management

All architectural state is declared in `CPUState`. No hidden globals or implicit state.

2. Deterministic Execution

Given an identical initial state and program, execution is perfectly reproducible.

3. Fault-Safe Design

Faults halt execution cleanly without corrupting architectural state. The previous design allowed partial updates during division by zero; this version treats faults atomically.

4. Observational Metrics

Cycle counting is a read-only metric, never used for control decisions. This separates performance observation from correctness.

5. Instruction Ownership

Each instruction explicitly controls PC advancement rather than relying on a central post-increment rule with exceptions. Instruction fields may be wider than their currently decoded meaning; unused bits are explicitly reserved to simplify decoding and allow future ISA extension.

Example Programs

Program 1: Simple Arithmetic

```
load_program(cpu, {
    0x100A,  // R0 = 10
    0x1105,  // R1 = 5
    0x2001,  // R0 = R0 + R1  (R0 = 15)
    0xF000   // HALT
});
```

Program 2: Memory Store

```
load_program(cpu, {
    0x100A,  // R0 = 10
    0x1105,  // R1 = 5
    0x2001,  // R0 = R0 + R1
    0x3002,  // MEM[R2] = R0  (store 15 at address 0)
    0xF000   // HALT
});
```

Program 3: Division by Zero (Fault)

```
load_program(cpu, {
    0x100A,  // R0 = 10
    0x1100,  // R1 = 0
    0x6001,  // R0 = R0 / R1  → FAULT
    0xF000   // Never reached
});
// Result: HaltReason::DIVISION_BY_ZERO
```

Program 4: Infinite Loop

```
load_program(cpu, {
    0x1001,  // R0 = 1
    0x7001   // JMP 1 (jump to self)
});
// Executes indefinitely
```

Testing Strategy

The implementation has been stress-tested with:

1. **Immediate halt**: Verifies minimal execution path.
2. **Arithmetic chains**: Tests multi-instruction sequences.
3. **Division by zero**: Confirms fault handling.
4. **Backward jumps**: Test PC manipulation.
5. **Infinite loops**: Validates sustained execution.
6. **Memory overwrites**: Tests self-modifying code behaviour.
7. **Unknown opcodes**: Verifies error detection.
8. **Faults in loops**: Tests fault dominance over loop control.
9. **PC boundary conditions**: Test execution at memory limits.

Build and Run

Compilation

```
g++ -std=c++17 -Wall -Wextra -O2 main.cpp -o cpu_interpreter
```

Execution

```
./cpu_interpreter
```

Requirements

- C++17 or later
- Standard library support for `<array>`, `<vector>`, `<iostream>`

Implementation Notes

Why `uint16_t` Registers?

Using 16-bit registers (rather than 8-bit or 32-bit) provides:

- Natural mapping to 16-bit instruction words.
- Full 64K address space coverage via register-indirect addressing.
- Avoids overflow edge cases in example programs.

Why Unified Memory?

The Harvard architecture (separate instruction/data memory) is omitted for simplicity. Real CPUs cache and prefetch differently, but this interpreter prioritises clarity.

Why Suppress Default PC Increment?

Rather than always incrementing PC and then overwriting it for jumps, each instruction declares whether it wants the default increment. This makes control flow explicit.

Future Extensions

Potential enhancements:

- **More instructions**: Bitwise operations, conditional jumps.
- **Interrupt support**: External event handling.
- **Pipeline simulation**: Model instruction overlap.
- **Cache simulation**: Add memory hierarchy.
- **Assembler**: Human-readable assembly language.
- **Debugger interface**: Breakpoints, single-stepping.

This interpreter was designed to explore clean architectural state management and deterministic fault handling. The previous implementation allowed silent partial state corruption during division by zero; this version treats all faults as atomic architectural events.

The observational cycle counter demonstrates how to add performance metrics without coupling them to correctness logic, a principle applicable to any interpreter or emulator design.